/ SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _Lavanya.M(192321051)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

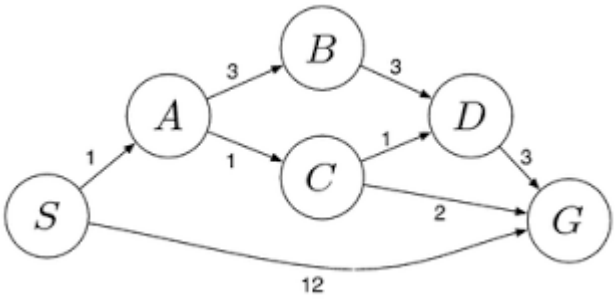
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

1. Water Jug Problem

Problem

We have:

- One **4-gallon jug**
- One **3-gallon jug**
- No measuring marks
- Unlimited water from a pump

Goal: Obtain exactly **2 gallons in the 4-gallon jug**.

State Representation

Represent a state as:

(x, y)

where:

- x = amount of water in 4-gallon jug
- y = amount of water in 3-gallon jug

Initial state:

(0, 0)

Goal state:

(2, y)

where the 4-gallon jug contains exactly 2 gallons.

Steps

Step	Action	4-Gallon	3-Gallon
1	Start with both empty	0	0
2	Fill the 3-gallon jug	0	3
3	Pour 3-gallon jug into 4-gallon jug	3	0
4	Fill the 3-gallon jug again	3	3
5	Pour 3-gallon jug into 4-gallon jug until 4-gallon is full	4	2
6	Empty the 4-gallon jug onto the ground	0	2
7	Pour the remaining 2 gallons into the 4-gallon jug	**2**	0

Final State

(2, 0)

Therefore, the **4-gallon jug contains exactly 2 gallons**.

Result

The Water Jug problem is solved in **7 operations**, obtaining exactly **2 gallons in the 4-gallon jug** without using any measuring device.

2. Mars Rover Intelligent Agent

A Mars Rover is an intelligent agent that observes its environment, makes decisions, moves around Mars, collects samples and analyzes them.

1. What are the percepts for this agent?

Percepts are the information received by the rover through its sensors.

The rover can perceive:

- Terrain and rocks
- Obstacles
- Soil composition
- Temperature
- Atmospheric conditions
- Position and location
- Sample characteristics

- Images of the Martian surface
- Battery/power level
- Distance from target
- Possible hazards

Example
If the rover's camera detects a large rock in front of it, the rover perceives:
"Obstacle detected ahead."

2. Characterize the Operating Environment
The Mars Rover environment can be characterized as:

Property	**Mars Rover Environment**
Observability	**Partially observable**
Number of agents	**Single agent**
Deterministic?	**Partially stochastic**
Episodic/Sequential	**Sequential**
Static/Dynamic	**Dynamic**
Discrete/Continuous	**Continuous**
Known/Unknown	**Unknown/partially known**

Explanation
- **Partially observable:** Rover cannot see the entire planet at once.
- **Single agent:** The rover generally operates independently.
- **Stochastic:** Terrain, weather and unexpected events can affect actions.
- **Sequential:** A current decision affects future decisions.
- **Dynamic:** Environmental conditions can change.
- **Continuous:** Position, distance, temperature, etc. vary continuously.
- **Unknown:** The rover discovers new terrain and information during exploration.

3. What actions can the agent take?
The rover can:
1. Move forward
2. Move backward
3. Turn left
4. Turn right
5. Stop
6. Collect soil samples
7. Collect rock samples
8. Drill the surface
9. Analyze samples
10. Capture images
11. Send data to Earth
12. Recharge/manage power
13. Avoid obstacles

4. How can we evaluate the performance?
The rover's performance can be measured using a **performance measure**.
Important measures include:
- Number of useful samples collected
- Accuracy of sample analysis

- Distance explored
- Scientific discoveries
- Successful navigation
- Amount of useful data transmitted
- Energy consumption
- Mission completion
- Avoidance of obstacles and damage
- Time taken to complete objectives

Performance Measure

A good rover should:

Maximize scientific information and exploration while minimizing energy consumption, time and risk.

5. Suitable Agent Architecture

The most suitable architecture is a **Goal-Based Agent with Utility-Based and Learning capabilities**.

Why?

The rover has specific goals such as:

- Reach a particular location
- Collect samples
- Analyze samples
- Avoid obstacles
- Conserve energy

A **goal-based agent** helps decide actions that achieve these goals.

A **utility-based component** can select the best action by considering:

- Scientific value
- Energy
- Time
- Safety
- Distance

A **learning component** can improve its behavior from previous exploration.

Final Answer

Best architecture: Hybrid Goal-Based + Utility-Based + Learning Agent.

3. 8-Queens Problem

Problem

The objective is to place **8 queens on an 8 × 8 chessboard** such that **no two queens attack each other**.

A queen can attack another queen if they are:

- In the same row
- In the same column
- On the same diagonal

Problem Formulation

1. Initial State

The chessboard starts with **no queens placed**.

Initial state = empty board

2. State

A state represents a partial or complete placement of queens.

For example:
[1, 5, 8, 6, 3, 7, 2, 4]
can represent the row positions of queens in columns 1 to 8.

3. Actions
An action is:
Place a queen in an empty square where it does not conflict with previously placed queens.

4. Transition Model
After placing a queen, the board changes from the current state to a new state containing one additional queen.
Example:
Empty board → 1 queen → 2 queens → ... → 8 queens

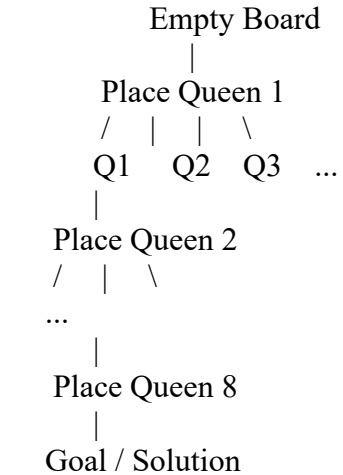
5. Goal Test
The goal is reached when:

- 8 queens are placed
- No two queens share the same row
- No two queens share the same column
- No two queens share a diagonal

Therefore:
Goal = 8 non-attacking queens

6. Path Cost
For the basic 8-Queens problem, every queen placement can have a cost of **1**.
Therefore, a complete solution has:
Path cost = 8

Search Space
The search starts from the empty board and explores possible configurations.
A simple search tree is:



Invalid configurations are discarded when queens attack each other.

Search Strategies
The problem can be explored using:
1. Breadth First Search (BFS)

Explores all states at one level before moving to the next level.

2. Depth First Search (DFS)

Explores one configuration deeply before backtracking.

3. Backtracking

Backtracking is particularly suitable for 8-Queens because whenever a queen creates a conflict, we immediately remove it and try another position.

Result

The 8-Queens problem is formulated as a **state-space search problem**, where the goal is to find a board configuration containing 8 non-attacking queens.

4. OLA Cab Booking Problem

Problem

A customer wants to travel from a source location to a destination using an OLA-like cab booking application.

Available cab types may include:

- Mini
- Micro
- Sedan
- Shared
- Prime

The customer selects a cab based on factors such as:

- Cost
- Comfort
- Travel time
- Availability
- Distance
- Preference

Type of Problem-Solving Agent

The most suitable agent is a:

Utility-Based Agent

Why?

The agent does not simply ask:

"Can I reach the destination?"

Instead, it compares different choices and selects the **best option** based on multiple factors.

For example:

Cab	Cost	Comfort	Time
Micro	Low	Medium	Medium
Mini	Medium	Medium	Medium
Sedan	High	High	Medium
Prime	Very High	Very High	Fast
Shared	Very Low	Low	Longer

The utility function can consider:

Utility = Comfort + Convenience + Speed − Cost

The agent chooses the cab with the highest utility according to the customer's preferences.

Pseudocode
START

Input source location
Input destination location

Find available cabs

For each available cab:
  Calculate fare
  Calculate estimated travel time
  Check comfort level
  Check availability
  Calculate utility value

Select cab with highest utility

Display cab type, fare and estimated time

If customer accepts:
  Confirm booking
  Assign driver
  Start trip
Else:
  Display other available options

END
Result
A **utility-based problem-solving agent** is appropriate because it evaluates multiple alternatives and selects the cab that provides the best overall utility according to the customer's requirements.

5. Uniform Cost Search for Logistics Network
Problem
A logistics company needs to find the **least-cost path from starting warehouse S to destination G**.
Each edge in the graph represents:
Travel cost = fuel cost + time cost
The appropriate algorithm is:
Uniform Cost Search (UCS)
UCS always expands the node having the **lowest cumulative path cost** from the starting node.
UCS Principle
If:
S → A = 4
S → B = 2
UCS chooses **B first**, because:
Cost(S,B) = 2 < Cost(S,A) = 4
It continues expanding the lowest-cost path until it reaches **G**.

UCS Algorithm

START

Put S into the priority queue with cost 0

WHILE priority queue is not empty:

  Remove node with the lowest path cost

  IF node is G:
    Return the path and total cost

  For each neighboring node:
    Calculate new path cost

    IF the node is not visited
    OR new cost is smaller:
      Add/update node in priority queue

END

UCS Steps

1. Start from **S** with cumulative cost **0**.
2. Expand the node having the smallest cumulative cost.
3. Add its neighboring warehouses to the priority queue.
4. Continue selecting the lowest-cost path.
5. When **G** is selected for expansion, the least-cost path has been found.

Important Point

UCS does **not** choose the edge with the smallest individual cost.
It chooses the node with the smallest:
Total cumulative cost from S
For example:
S → A → G

Cost = Cost(S,A) + Cost(A,G)
and
S → B → G

Cost = Cost(S,B) + Cost(B,G)
UCS compares the **total costs** and chooses the minimum.
From the given graph, the edge costs are:

- S → A = 1
- S → G = 12
- A → B = 3
- A → C = 1
- B → D = 3
- C → D = 1
- C → G = 2
- D → G = 3

Goal

Find the **least-cost path from S to G** using Uniform Cost Search.

Step 1: Start from S
Initial cost:
S = 0
From S:

- S → A = **1**
- S → G = **12**

Choose the lowest cost: **A (1)**.

Step 2: Expand A
From A:

- A → B = 3
  Total cost = 1 + 3 = **4**
- A → C = 1
  Total cost = 1 + 1 = **2**

Priority queue:

Node	Path	Total Cost
C	S → A → C	**2**
B	S → A → B	4
G	S → G	12

Choose **C (2)**.

Step 3: Expand C
From C:

- C → D = 1
  Total = 2 + 1 = **3**
- C → G = 2
  Total = 2 + 2 = **4**

Priority queue:

Node	Path	Total Cost
D	S → A → C → D	**3**
B	S → A → B	4
G	S → A → C → G	**4**
G	S → G	12

Choose **D (3)**.

Step 4: Expand D
From D:

- D → G = 3

Total cost:
3 + 3 = 6
The queue now contains:

Node	Path	Total Cost
B	S → A → B	4
G	S → A → C → G	**4**
G	S → A → C → D → G	6
G	S → G	12

The lowest-cost node is **G with cost 4**.
Therefore, UCS terminates.

UCS Search Table

Step	Expanded Node	Cumulative Cost	New Nodes
1	S	0	A(1), G(12)
2	A	1	C(2), B(4)
3	C	2	D(3), G(4)
4	D	3	G(6)
5	G	**4**	Goal reached

Final Answer
Least-cost path:
S → A → C → G
Total minimum cost:

$$1 + 1 + 2 = \boxed{4}$$

Result:
Using **Uniform Cost Search**, the logistics company should transport the package through:
S → A → C → G
with a **minimum total travel cost of 4**.